

# EXAM ROOM ALLOCATION SYSTEM

## PROJECT REPORT

|                      |                                               |
|----------------------|-----------------------------------------------|
| <b>Hackathon:</b>    | AlgoForge                                     |
| <b>Team Members:</b> | A Harini Reddy<br>M Deeksha<br>K Rakshitha    |
| <b>Institution:</b>  | Malla Reddy Vishwavidyapeeth Technical Campus |
| <b>Date:</b>         | 03-09-2026                                    |

## 2. ABSTRACT & INTRODUCTION

### 2.1 Abstract

The Exam Room Allocation System is a web-based application developed to simplify and organize examination management in educational institutions. Conducting examinations involves maintaining student records, managing examination details, assigning rooms, preparing seating arrangements, generating timetables, and recording attendance. Manual processes can require significant time and may result in duplicate allocations, scheduling conflicts, incorrect records, and inefficient use of examination rooms.

The system provides a centralized platform for student management, examination management, room management, room allocation, timetable generation, timetable conflict checking, seating arrangements, student search, attendance marking, an attendance dashboard, and PDF report downloads.

The application uses Python and Flask for backend processing, HTML and CSS for the user interface, SQLite for database management, and ReportLab for PDF generation. The main objective is to reduce repetitive manual work and improve the organization of examination records.

### 2.2 Introduction

Examinations are an essential component of the academic system. Institutions must assign students to suitable rooms, respect room capacities, organize examination schedules, and record attendance accurately.

Traditional room allocation often involves spreadsheets, paper records, and manual calculations. As student and examination counts increase, these tasks become more difficult. Errors may include over-capacity room assignments, incorrect seating information, and overlapping examination schedules.

The Exam Room Allocation System addresses these challenges through a web-based application. Administrators can manage student information, examination details, room information, seating arrangements, timetables, and attendance. The application stores relevant information in SQLite and supports PDF report downloads.

## 3. PROBLEM STATEMENT, OBJECTIVES & SCOPE

### 3.1 Problem Statement

Examination room allocation requires coordination between students, examination schedules, and available rooms. Manual processes require administrators to review student lists, calculate room capacities, assign students, prepare seating plans, and verify timetable conflicts. These activities may cause duplicate allocations, incorrect seating information, scheduling conflicts, inefficient use of seats, and difficulties in maintaining attendance records.

A computerized system is therefore needed to centralize examination information, organize room allocation, support timetable preparation, and simplify related administrative activities.

### 3.2 Objectives

- Develop a web-based application for examination room allocation.
- Maintain student information in a structured database.
- Manage examination details and schedules.
- Maintain examination room information and seating capacities.
- Assign students according to available room capacity.
- Generate examination timetables and identify conflicts.
- Prepare and display seating arrangement information.
- Provide student search functionality.
- Record and display student attendance through an attendance dashboard.
- Generate downloadable PDF reports.

- Reduce repetitive administrative work and improve access to examination records.

### 3.3 Scope

#### Included in the current system:

- Student record management
- Examination record management
- Examination room management
- Room allocation based on capacity
- Timetable generation and conflict checking
- Seating arrangement display
- Student search
- Attendance marking and attendance dashboard
- PDF report generation and downloads
- Database storage and retrieval

**Scope boundaries:** Automated notifications, student self-service login, QR-code attendance, cloud deployment, and integration with external institutional systems are possible future enhancements rather than confirmed features of the current version.

## 4. FEATURES & FUNCTIONAL REQUIREMENTS

### 4.1 Student Management

Maintains student records required for examination planning and room allocation, and supports retrieval of student information.

### 4.2 Examination Management

Organizes examination information entered by the administrator, including dates and timings where supported by the application.

### 4.3 Room Management

Maintains room details and seating capacities, helping administrators check whether sufficient seats are available.

### 4.4 Exam Room Allocation

Supports assigning students to examination rooms using student records, examination details, and room capacity. Remaining students should be identified if capacity is insufficient.

### 4.5 Timetable Generation

Generates and presents examination schedules using the examination information entered into the application.

### 4.6 Timetable Conflict Checking

Helps identify overlapping schedules, such as examinations assigned to the same room at overlapping times or examinations involving the same student, depending on the implemented checks.

### 4.7 Seating Arrangement

Displays student allocation information by examination room to help administrators review seating arrangements.

### 4.8 Student Search

Helps administrators locate student records without manually reviewing the full list.

## 4.9 Attendance Management

Supports marking student attendance and storing attendance information for review.

## 4.10 Attendance Dashboard

Provides a summarized view of attendance information stored in the system.

## 4.11 PDF Report Downloads

Allows supported examination information to be downloaded in PDF format using ReportLab.

## 4.12 Functional Requirements

| ID   | Functional requirement                                 |
|------|--------------------------------------------------------|
| FR1  | Support student record management.                     |
| FR2  | Support examination record management.                 |
| FR3  | Maintain room information and capacities.              |
| FR4  | Support examination room allocation.                   |
| FR5  | Consider room capacity during allocation.              |
| FR6  | Support timetable generation.                          |
| FR7  | Identify relevant timetable conflicts.                 |
| FR8  | Display seating arrangement information.               |
| FR9  | Support student search.                                |
| FR10 | Support attendance marking.                            |
| FR11 | Provide an attendance dashboard.                       |
| FR12 | Support PDF report downloads.                          |
| FR13 | Store and retrieve application records through SQLite. |

## 4.13 Non-Functional Requirements

- **Usability:** The interface should be easy to understand and navigate.
- **Reliability:** Records should be stored and retrieved consistently.
- **Performance:** Common operations should complete within a reasonable time for the stored data.
- **Maintainability:** The application should allow individual modules to be updated.
- **Data integrity:** Allocation and timetable operations should respect capacity and scheduling constraints.
- **Compatibility:** The interface should work in a compatible modern browser.

# 5. ALGORITHMS & METHODOLOGY

## 5.1 Overall Methodology

The system follows a workflow that begins with maintaining records and ends with producing examination arrangements and reports.

- Enter student information.
- Enter examination details.
- Maintain room information.

- Retrieve relevant students, examinations, and rooms.
- Check room availability and capacity.
- Check timetable constraints and potential conflicts.
- Allocate students to suitable rooms.
- Display seating arrangement information.
- Generate and review examination timetables.
- Mark attendance and review the attendance dashboard.
- Generate PDF reports when required.

The precise allocation order and conflict rules depend on the implemented application logic.

## 5.2 Room Allocation Algorithm

**Objective:** Assign students to examination rooms without exceeding available seating capacity.

**Inputs:** Students requiring allocation, examination details, rooms, room capacities, and availability information where supported.

- Retrieve students requiring allocation.
- Retrieve available examination rooms.
- Read each room's capacity.
- Check room availability for the examination.
- Select an appropriate room.
- Assign students while seats remain available.
- Record each allocation.
- Continue until all students are assigned or no suitable seats remain.
- Report if additional capacity is required.
- Display the allocation results.

**Pseudocode:**

```

START
Retrieve students, examination details, and available rooms
FOR each room: check availability and read capacity
WHILE seats and unallocated students remain: assign next student and save allocation
IF students remain: display insufficient capacity message
Display allocation results
STOP
  
```

This pseudocode explains the general method; it is not a claim that the source code uses this exact sequence.

## 5.3 Capacity Validation

Let  $S$  be the number of students requiring allocation,  $C_i$  the capacity of room  $i$ , and  $C_{total}$  the total available capacity.

$$C_{total} = \sum C_i$$

If  $S \leq C_{total}$ , total capacity is sufficient in principle, provided rooms are available at the required time. If  $S > C_{total}$ , additional rooms or another arrangement is required.

## 5.4 Timetable Conflict Checking

The system can check examination dates and time intervals and flag overlapping schedules that violate room or student constraints, depending on the checks implemented.

For examinations on the same date, intervals overlap when  $Start_1 < End_2$  and  $Start_2 < End_1$ .

## 5.5 Seating Arrangement Method

Retrieve allocation records, group them by room, display the relevant student information for each room, and allow the administrator to review the arrangement.

## 5.6 Student Search Method

Accept the search value, process the request in the backend, retrieve matching database records, and display the results. The available search fields depend on the application.

## 5.7 Attendance Processing Method

Retrieve the relevant student or examination records, select the attendance status, save the record, retrieve attendance information for the dashboard, and display the updated information.

## 5.8 PDF Generation Method

Retrieve the required report information, organize it, create the PDF using ReportLab, add supported headings and record details, and make the document available for download.

## 5.9 Algorithmic Complexity

Exact complexity depends on the source code and data structures. A simple allocation method that processes each student once may have approximately linear assignment work, excluding database and room-selection overhead. A straightforward conflict checker comparing every pair of  $E$  examinations takes  $O(E^2)$  time. These are general estimates, not measured benchmarks or verified descriptions of the exact implementation.

# 6. TECHNOLOGIES & IMPLEMENTATION

## Python

Used for backend logic and data processing.

## Flask

A Python web framework used to define routes, receive requests, process input, and return pages or downloadable documents.

## HTML

Used to structure headings, forms, tables, and other interface elements.

## CSS

Used to style the interface, including layout, spacing, and readability.

## SQLite

Used to store and retrieve application records. The database file is named exam\_allocation.db.

## ReportLab

A Python library used to generate PDF documents from application data.

**Database safety:** Preserve the existing exam\_allocation.db file because it may contain the project's saved data.

## 6.1 System Architecture

**Presentation layer:** HTML and CSS allow administrators to enter information, access modules, view records, and download reports.

**Application layer:** Flask handles requests and application logic, connects the interface to the database, and processes record management, allocation, timetable, attendance, and PDF operations.

**Database layer:** SQLite stores and retrieves application records.

**Workflow:** Administrator → HTML/CSS Interface → Flask Backend → SQLite Database and application logic → Results or PDF downloads.

## 6.2 Database Implementation

SQLite provides persistent storage for application records. The database supports student, examination, room, allocation, and attendance workflows as represented in the implemented schema. Exact table names, columns, keys, and relationships should be documented from the actual schema rather than assumed.

## 6.3 Module Implementation

### Student Module

Provides access to student records and supports allocation and search.

### Examination Module

Maintains examination information used for timetable generation and room allocation.

### Room Module

Stores room details and seating capacities.

### Allocation Module

Processes room assignments using student, examination, and room records.

### Timetable Module

Supports schedule generation and relevant conflict checking.

### Seating Module

Displays room allocation information.

### Search Module

Processes search requests and displays matching student records.

### Attendance Module

Supports attendance marking and dashboard information.

### PDF Module

Uses ReportLab to generate supported downloadable reports.

## 6.4 Application Workflow

The administrator enters or maintains information through the web interface. Flask processes requests and interacts with SQLite to store or retrieve records. The application displays the requested information and, where supported, uses ReportLab to generate PDF documents.

# 7. RESULTS, TESTING & PERFORMANCE

## 7.1 Results

The application brings together student management, examination management, room management, room allocation, timetable generation, conflict checking, seating arrangement display, student search, attendance marking, an attendance dashboard, and PDF downloads. It provides a structured way to manage examination records and review allocations. Accuracy depends on correct records, schedules, capacities, and application logic.

## 7.2 Testing Methodology

Testing should verify that each module accepts valid information, handles invalid or incomplete inputs, stores records correctly, and displays expected results. The following are test cases with expected outcomes; mark them as passed only after execution.

| Test ID | Test scenario                             | Expected result                                                  |
|---------|-------------------------------------------|------------------------------------------------------------------|
| T1      | Manage student records                    | Student information is saved and displayed.                      |
| T2      | Enter examination details                 | Examination information is available for planning.               |
| T3      | Add a room with capacity                  | Room details are stored correctly.                               |
| T4      | Allocate within capacity                  | Students are assigned without exceeding capacity.                |
| T5      | Insufficient room capacity                | Remaining students or an appropriate message is shown.           |
| T6      | Generate timetable                        | Timetable is displayed using entered details.                    |
| T7      | Overlapping schedules in same room        | Room conflict is identified if covered by implemented checks.    |
| T8      | Overlapping examinations for same student | Student conflict is identified if covered by implemented checks. |
| T9      | View seating arrangements                 | Student and room allocation information is displayed.            |
| T10     | Search for a student                      | Matching records are displayed.                                  |
| T11     | Mark attendance                           | Attendance information is saved and reviewable.                  |
| T12     | Open attendance dashboard                 | Available attendance information is displayed.                   |
| T13     | Download a PDF                            | A valid PDF is generated and downloaded.                         |
| T14     | Restart application and retrieve records  | Persisted records remain in the database.                        |

## 7.3 Performance Analysis

Flask handles web requests and SQLite provides lightweight data storage. Actual response time depends on the number of students, examinations, rooms, database queries, and concurrent users. Performance should be evaluated with representative data and measured operation times. No measured response times or benchmark results are claimed here.

## 7.4 Reliability and Data Integrity

- Ensure allocations do not exceed room capacity.
- Check relevant timetable overlaps.
- Confirm attendance records are saved.
- Verify student and examination records can be retrieved.
- Confirm generated PDFs open correctly.
- Preserve the existing database file.

## 7.5 Limitations

- Allocation results depend on accurate records and the implemented logic.
- Conflict checking depends on the constraints covered by the code.
- Advanced seat optimization is not assumed to be available.
- Automated notifications are not included in the current version.
- Large-scale performance has not been benchmarked.